JADAVPUR UNIVERSITY

188, Raja S.C. Mallick Road, Kolkata – 700032

Department of Computer Science and Engineering, FET.

MASTER OF COMPUTER APPLICATION

1st Year 2nd Semester

JAVA ASSIGNMENT

Name: Snehasish Sarkar
Roll No.: 002510503033

Assignment 1: Write a java program with a special class

- **To store name and dob**
- **To calculate age**
- **To specify age category as follows:**
  - **"Under Age", if Age < 18**
  - **"Young Adult" if 18<age<30**
  - **"Matured Adult", if 30<age<45**
  - **"Middle Aged" if 45<age<60 & so on.**

Source Code:

```
import java.time.LocalDate;
import java.time.Period;
import java.util.Scanner;

class Person {
    private String name;
    private LocalDate dob;
    private int age;


    Person(String name, LocalDate dob) {
        this.name = name;
        this.dob = dob;
        this.age = calculateAge();
    }

    private int calculateAge() {
        LocalDate today = LocalDate.now();
        return Period.between(dob, today).getYears();
    }

    public String getAgeCategory() {
        if (age < 18)
            return "Under Age";
        else if (age >= 18 && age < 30)
            return "Young Adult";
        else if (age >= 30 && age < 45)
            return "Matured Adult";
        else if (age >= 45 && age < 60)
            return "Middle Aged";
        else
            return "Senior Citizen";
    }

    public String toString() {
        return "\nName : "+name+"\nDOB : "+dob+"\nAge : "+age+"\nCategory :
"+getAgeCategory();
    }
}

public class AgeCategory {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter Name: ");
        String name = sc.nextLine();

        System.out.print("Enter Birth Year (yyyy): ");
        int year = sc.nextInt();

        System.out.print("Enter Birth Month (mm): ");
        int month = sc.nextInt();

        System.out.print("Enter Birth Day (dd): ");
        int day = sc.nextInt();

        LocalDate dob = LocalDate.of(year, month, day);

        Person p = new Person(name, dob);
        System.out.println(p);

        sc.close();
    }
}
```

Output :

```
Enter Name: Snehasish Sarkar
Enter Birth Year (yyyy): 2004
Enter Birth Month (mm): 07
Enter Birth Day (dd): 23

Name : Snehasish Sarkar
DOB : 2004-07-23
Age : 21
Category : Young Adult
```

Assignment 2: Design you own Stack such that the stack class objects need to be :

- **Initialized by a constructor**
- **Provided proper push() and pop() methods**
- **Checked for tos content using peek()**
- **Overload with display() for several tasks**
  - **Display content of the whole stack**
  - **Give a parameter for the depth of a particular element, display just that element.**

Source Code :

```
import java.util.Scanner;

class MyStack {
    private int[] stack;
    private int top;
    private int size;
    public MyStack(int size) {
        this.size = size;
        stack = new int[size];
        top = -1;
    }
    public void push(int value) {
        if (top == size - 1) {
            System.out.println("Stack Overflow!");
            return;
        }
        stack[++top] = value;
        System.out.println(value + " pushed into stack.");
    }
    public void pop() {
        if (top == -1) {
            System.out.println("Stack Underflow!");
            return;
        }

        System.out.println("Popped Element: " + stack[top--]);
    }

    public void peek() {
        if (top == -1) {
            System.out.println("Stack is Empty!");
            return;
        }

        System.out.println("Top Element: " + stack[top]);
    }
    public void display() {
        if (top == -1) {
            System.out.println("Stack is Empty!");
            return;
        }

        System.out.println("\nStack Contents (Top to Bottom):");
        for (int i = top; i >= 0; i--) {
            System.out.println(stack[i]);
```

```
        }
    }
    public void display(int depth) {
        if (top == -1) {
            System.out.println("Stack is Empty!");
            return;
        }
        if (depth < 0 || depth > top) {
            System.out.println("Invalid Depth!");
            return;
        }
        System.out.println("Element at Depth " + depth +
                           " : " + stack[top - depth]);
    }
}

public class Stack{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Stack Size: ");
        int size = sc.nextInt();
        MyStack stack = new MyStack(size);

        int choice;
        do {
            System.out.println("\n===== STACK MENU =====");
            System.out.println("1. Push");
            System.out.println("2. Pop");
            System.out.println("3. Peek");
            System.out.println("4. Display Entire Stack");
            System.out.println("5. Display Element at Given Depth");
            System.out.println("6. Exit");
            System.out.print("Enter Choice: ");
            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    System.out.print("Enter Element: ");
                    int value = sc.nextInt();
                    stack.push(value);
                    break;

                case 2:
                    stack.pop();
                    break;

                case 3:
                    stack.peek();
                    break;

                case 4:
                    stack.display();
                    break;
```

```
                case 5:
                    System.out.print("Enter Depth: ");
                    int depth = sc.nextInt();
                    stack.display(depth);
                    break;

                case 6:
                    System.out.println("Exiting...");
                    break;

                default:
                    System.out.println("Invalid Choice!");
            }

        } while (choice != 6);

        sc.close();
    }
}
```

Output :

```
Enter Stack Size: 2

===== STACK MENU =====
1. Push
2. Pop
3. Peek
4. Display Entire Stack
5. Display Element at Given Depth
6. Exit
Enter Choice: 1
Enter Element: 10
10 pushed into stack.

===== STACK MENU =====
1. Push
2. Pop
3. Peek
4. Display Entire Stack
5. Display Element at Given Depth
6. Exit
Enter Choice: 1
Enter Element: 20
20 pushed into stack.

===== STACK MENU =====
1. Push
2. Pop
3. Peek
4. Display Entire Stack
5. Display Element at Given Depth
6. Exit
Enter Choice: 1
Enter Element: 30
Stack Overflow!
```

```
===== STACK MENU =====
1. Push
2. Pop
3. Peek
4. Display Entire Stack
5. Display Element at Given Depth
6. Exit
Enter Choice: 3
Top Element: 20

===== STACK MENU =====
1. Push
2. Pop
3. Peek
4. Display Entire Stack
5. Display Element at Given Depth
6. Exit
Enter Choice: 4

Stack Contents (Top to Bottom):
20
10

===== STACK MENU =====
1. Push
2. Pop
3. Peek
4. Display Entire Stack
5. Display Element at Given Depth
6. Exit
Enter Choice: 5
Enter Depth: 1
Element at Depth 1 : 10
```

```
===== STACK MENU =====
1. Push
2. Pop
3. Peek
4. Display Entire Stack
5. Display Element at Given Depth
6. Exit
Enter Choice: 2
Popped Element: 20

===== STACK MENU =====
1. Push
2. Pop
3. Peek
4. Display Entire Stack
5. Display Element at Given Depth
6. Exit
Enter Choice: 2
Popped Element: 10

===== STACK MENU =====
1. Push
2. Pop
3. Peek
4. Display Entire Stack
5. Display Element at Given Depth
6. Exit
Enter Choice: 2
Stack Underflow!

===== STACK MENU =====
1. Push
2. Pop
3. Peek
4. Display Entire Stack
5. Display Element at Given Depth
6. Exit
Enter Choice: 6
```

Assignment 3:

- **Implement merge operations on two stacks presorted in ascending order. Check for stack overflow conditions all along.**
- **Inherit a special Stack**
  - **The inherited class should allow : A number to be pushed in if and only if it is less than the one currently at the top.**
  - **With the help of such STACK objects, solve the Tower of Hanoi problem most economically, by arranging a sequence of haphazard numbers in the original base type Stack in ascending order in a final STACK.**

Source Code :

```
import java.util.Scanner;

class StackException extends Exception {
    public StackException(String msg) {
        super(msg);
    }
}

class Stack {
    protected int[] stk;
    protected int top;
    protected int size;

    public Stack(int size) {
        this.size = size;
        stk = new int[size];
        top = -1;
    }

    public boolean isEmpty() {
        return top == -1;
    }

    public boolean isFull() {
        return top == size - 1;
    }

    public void push(int item) throws StackException {
        if (isFull())
            throw new StackException("Stack Overflow");

        stk[++top] = item;
    }

    public int pop() throws StackException {
        if (isEmpty())
            throw new StackException("Stack Underflow");

        return stk[top--];
    }

    public int peek() throws StackException {
```

```
        if (isEmpty())
            throw new StackException("Stack Empty");

        return stk[top];
    }

    public void display() {
        if (isEmpty()) {
            System.out.println("Stack Empty");
            return;
        }

        System.out.println("Top -> Bottom");

        for (int i = top; i >= 0; i--)
            System.out.println(stk[i]);
    }
}

class SpecialStack extends Stack {

    public SpecialStack(int size) {
        super(size);
    }

    @Override
    public void push(int item) throws StackException {

        if (isFull())
            throw new StackException("Stack Overflow");

        if (!isEmpty() && item >= stk[top])
            throw new StackException(
                    "Cannot place larger disk on smaller disk");

        stk[++top] = item;
    }
}

public class Tower {

    static SpecialStack t1;
    static SpecialStack t2;
    static SpecialStack t3;
    static int moves = 0;


    static Stack merge(Stack s1, Stack s2) throws StackException {
        Stack temp = new Stack(s1.size + s2.size);
        Stack result = new Stack(s1.size + s2.size);

        while (!s1.isEmpty() && !s2.isEmpty()) {
```

```
            if (s1.peek() > s2.peek())
                temp.push(s1.pop());
            else
                temp.push(s2.pop());
        }

        while (!s1.isEmpty())
            temp.push(s1.pop());

        while (!s2.isEmpty())
            temp.push(s2.pop());

        while (!temp.isEmpty())
            result.push(temp.pop());

        return result;
    }
    static int valueAt(SpecialStack s, int index) {
        if (index <= s.top)  return s.stk[index];
        return 0;
    }

    static void displayTowers() {
        int start = Math.max(t1.top,  Math.max(t2.top, t3.top));

        System.out.println();
        for (int i = start; i >= 0; i--) {
            System.out.printf( "| %2d |   | %2d |   | %2d |\n", valueAt(t1, i),
valueAt(t2, i), valueAt(t3, i));

            System.out.println("--------------------------");
        }

        System.out.println("   A        B        C");
        System.out.println();
    }
    static void hanoi(int disk, SpecialStack source,  SpecialStack auxiliary,
SpecialStack destination,  char s,  char a,  char d) throws StackException {
        if (disk == 1) {
            int item = source.pop();
            destination.push(item);
            moves++;
            System.out.println("Move Disk " + item + " from " + s + " to " + d);

            displayTowers();
            return;
        }
        hanoi(disk - 1, source,  destination, auxiliary,  s, d, a);

        int item = source.pop();
        destination.push(item);
        moves++;
        System.out.println("Move Disk " + item + " from " + s +  " to " + d);
```

```
        displayTowers();
        hanoi(disk - 1, auxiliary, source, destination, a, s, d);
    }


    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int choice;

        do {
            System.out.println("\n===== ASSIGNMENT 3 =====");
            System.out.println("1. Merge Two Sorted Stacks");
            System.out.println("2. Tower Of Hanoi");
            System.out.println("3. Exit");
            System.out.print("Enter Choice : ");
            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    try {
                        System.out.print("Size of Stack 1 : ");
                        int n1 = sc.nextInt();
                        Stack s1 = new Stack(n1);
                        System.out.println("Enter elements in ascending
order:");

                        for (int i = 0; i < n1; i++)
                            s1.push(sc.nextInt());

                        System.out.print("Size of Stack 2 : ");
                        int n2 = sc.nextInt();
                        Stack s2 = new Stack(n2);
                        System.out.println("Enter elements in ascending
order:");

                        for (int i = 0; i < n2; i++)
                            s2.push(sc.nextInt());

                        Stack merged = merge(s1, s2);
                        System.out.println("\nMerged Stack:");
                        merged.display();

                    } catch (StackException e) {
                        System.out.println(e.getMessage());
                    }

                    break;
                case 2:

                    try {
                        System.out.print("Enter number of disks : ");
                        int disk = sc.nextInt();
                        t1 = new SpecialStack(disk);
```

```
                        t2 = new SpecialStack(disk);
                        t3 = new SpecialStack(disk);
                        for (int i = disk; i >= 1; i--)
                            t1.push(i);

                        moves = 0;
                        System.out.println("\nInitial Towers:");
                        displayTowers();
                        hanoi(disk, t1, t2, t3,'A', 'B', 'C');
                        System.out.println("Total Moves = " + moves);
                        System.out.println("Calculated Moves = "+
((int)Math.pow(2, disk) - 1));

                    } catch (StackException e) {
                        System.out.println(e.getMessage());
                    }

                    break;

                case 3:
                    System.out.println("Program Ended.");
                    break;

                default:
                    System.out.println("Invalid Choice");
            }

        } while (choice != 3);
        sc.close();
    }
}
```

Output :

```
===== ASSIGNMENT 3 =====
1. Merge Two Sorted Stacks
2. Tower Of Hanoi
3. Exit
Enter Choice : 1
Size of Stack 1 : 3
Enter elements in ascending order:
10 20 30
Size of Stack 2 : 2
Enter elements in ascending order:
50 60

Merged Stack:
Top -> Bottom
60
50
30
20
10
```

```
===== ASSIGNMENT 3 =====
1. Merge Two Sorted Stacks
2. Tower Of Hanoi
3. Exit
Enter Choice : 2
Enter number of disks : 3
Initial Towers:

| 1 |   | 0 |   | 0 |
-------------------------
| 2 |   | 0 |   | 0 |
-------------------------
| 3 |   | 0 |   | 0 |
-------------------------
  A       B       C

Move Disk 1 from A to C

| 2 |   | 0 |   | 0 |
-------------------------
| 3 |   | 0 |   | 1 |
-------------------------
  A       B       C

Move Disk 2 from A to B

| 3 |   | 2 |   | 1 |
-------------------------
  A       B       C

Move Disk 1 from C to B

| 0 |   | 1 |   | 0 |
-------------------------
| 3 |   | 2 |   | 0 |
-------------------------
  A       B       C

Move Disk 3 from A to C

| 0 |   | 1 |   | 0 |
-------------------------
| 0 |   | 2 |   | 3 |
-------------------------
  A       B       C

Move Disk 1 from B to A

| 1 |   | 2 |   | 3 |
-------------------------
  A       B       C

Move Disk 2 from B to C

| 0 |   | 0 |   | 2 |
-------------------------
| 1 |   | 0 |   | 3 |
-------------------------
  A       B       C

Move Disk 1 from A to C

| 0 |   | 0 |   | 1 |
-------------------------
| 0 |   | 0 |   | 2 |
-------------------------
| 0 |   | 0 |   | 3 |
-------------------------
  A       B       C

Total Moves = 7
Calculated Moves = 7
```

Assignment 4:

- **Create a new class Graded_Emp from an existing class EMP, where class Emp got two fields ID and dep_code[0-5], whereas the derived class has an additional field Grade[A-E].**
- **Use the super keyword judiciously to**
  (a) Populate objects of both the base and derived classes above with minimum repetition of code; &
  (b) Change Dep_code 0 to 10 while populating derived objects.
- **Create your own exception class to trap accepting invalid data above.**
- **How would you restructure your code if the Grade field is dependent on average marks obtained in several skill-tests recorded separately?**

Source Code :

```
import java.util.Scanner;

class InvalidDataException extends Exception {
    public InvalidDataException(String msg) {
        super(msg);
    }
}

class EMP {
    protected int id;
    protected int depCode;

    public EMP(int id, int depCode)
            throws InvalidDataException {

        if (id <= 0)
            throw new InvalidDataException(
                    "Employee ID must be positive");

        if (depCode < 0 || depCode > 5)
            throw new InvalidDataException(
                    "Department Code must be between 0 and 5");

        this.id = id;
        this.depCode = depCode;
    }

    public void display() {
        System.out.println("Employee ID : " + id);
        System.out.println("Department Code : " + depCode);
    }
}

class GradedEmp extends EMP {

    private char grade;

    public GradedEmp(int id, int depCode,   char grade) throws
InvalidDataException {

        super(id, depCode == 0 ? 10 : depCode);
```

```
        grade = Character.toUpperCase(grade);
        if (grade < 'A' || grade > 'E')  throw new
InvalidDataException(  "Grade must be between A and E");
        this.grade = grade;
    }

    @Override
    public void display() {
        super.display();
        System.out.println("Grade : " + grade);
    }
}
public class Assignment4 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {

            System.out.print("Enter Employee ID : ");
            int id = sc.nextInt();
            System.out.print( "Enter Department Code (0-5) : ");
            int dep = sc.nextInt();

            System.out.print( "Enter Grade (A-E) : ");
            char grade = sc.next().charAt(0);

            GradedEmp emp =  new GradedEmp(id, dep, grade);

            System.out.println("\nEmployee Details");
            System.out.println("----------------");

            emp.display();

        } catch (InvalidDataException e) {
            System.out.println( "Exception : " + e.getMessage());
        } catch (Exception e) {
            System.out.println( "Invalid Input");
        }

        sc.close();
    }
}
```

Output :

```
Enter Employee ID : 101
Enter Department Code (0-5) : 2
Enter Grade (A-E) : A

Employee Details
----------------
Employee ID : 101
Department Code : 2
Grade : A
```

Assignment 5:

- **Create an interface Shape with the getArea() method. Create three classes Rectangle, Circle, and Triangle that implement the Shape interface. Implement the getArea() method for each of the classes.**
- **Create a Animal Interface with a method called sound() that takes no arguments and returns void. Create a few specific animal classes that implement Animal and overrides sound() differently for each species.**

Source Code :

```
package shapes;

public interface Shape{
    float getArea();
}

import shapes.Shape;

public class Circle implements Shape{
    float r;
    static float PI = 3.14f;
    Circle(float r) {
        this.r = r;
    }

    public float getArea() {
        return PI * 2 * r;
    }
}

import shapes.Shape;
public class Rectangle implements Shape{
    float l, b;
    Rectangle(float l, float b) {
        this.l = l;
        this.b = b;
    }

    public float getArea() {
        return this.l * this.b;
    }
}

import shapes.Shape;

public class Triangle implements Shape{
    float b, h;
    Triangle(float b, float h) {
        this.b = b;
        this.h = h;
    }

    public float getArea() {
        return 0.5f * this.b * this.h;
```

```
    }
}


public class Main {
    public static void main(String[] args) {
        Triangle t = new Triangle(10.0f, 20.0f);
        Circle c = new Circle(5f);
        Rectangle r = new Rectangle(12.0f, 13.0f);

        System.out.println("Area triangle   : "+t.getArea());
        System.out.println("Area circle     : "+c.getArea());
        System.out.println("Area rectangle  : "+r.getArea());
    }
}



public interface Animals {
    void sound();
}

public class Dog implements Animals{
    @Override
    public void sound() {
        System.out.println("Barks");
    }

    public static void main(String[] args) {
        Animals d = new Dog();
        d.sound();
    }
}
```

Output :

```
Area triangle   : 100.0
Area circle     : 31.400002
Area rectangle  : 156.0
Dog Barks
```

Assignment 6: Create an interface Sortable with a method sort() that sorts an array of integers in ascending order. Create a class BubbleSort that implements the Sortable interface and provide its own version of the sort method.

Source Code :

```
public interface Sortable {
    void sort(int[] arr);
}

public class BubbleSort implements Sortable {
    public void sort(int[] arr) {
        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < arr.length - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }

    void display(int[] arr) {
        System.out.print("\nSorted Array : | ");
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i]+" | ");
        }
    }


    public static void main(String[] args) {
        BubbleSort b = new BubbleSort();
        int[] arr = {10, 5, 20, 32, 1, 23, 42, 2, 1, 3, 4, 6};

        System.out.print("Unsorted Array : | ");
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i]+" | ");
        }

        b.sort(arr);

        b.display(arr);
    }
}
```

Output :

```
Unsorted Array : | 10 | 5 | 20 | 32 | 1 | 23 | 42 | 2 | 1 | 3 | 4 | 6 |
Sorted Array : | 1 | 1 | 2 | 3 | 4 | 5 | 6 | 10 | 20 | 23 | 32 | 42 |
```

Assignment 7:

- **Create an interface ResizableTray with methods W(int width) and L(int length) that help trays to be resized. Create a class Tray that implements the interface to carry a number of fixed size plates.**
- **Inherit an interface ResizableBox from ResizableTray with a new method D(int depth). Create a class NewBox that implements ResizableBox to house a given number of balls of fixed size using the overridden resizing methods.**

Source Code :

```
interface ResizableTray {

    void W(int width);

    void L(int length);
}

class Tray implements ResizableTray {

    protected int width;
    protected int length;
    protected int plateCount;

    public Tray(int width, int length, int plateCount) {
        this.width = width;
        this.length = length;
        this.plateCount = plateCount;
    }

    @Override
    public void W(int width) {
        this.width = width;
    }

    @Override
    public void L(int length) {
        this.length = length;
    }

    public void displayTray() {
        System.out.println("\nTray Details");
        System.out.println("Width = " + width);
        System.out.println("Length = " + length);
        System.out.println("Number of Plates = " + plateCount);
    }
}

interface ResizableBox extends ResizableTray {

    void D(int depth);
}

public class NewBox implements ResizableBox {

    private int width;
```

```
    private int length;
    private int depth;
    private int ballCount;

    public NewBox(int width,
                  int length,
                  int depth,
                  int ballCount) {

        this.width = width;
        this.length = length;
        this.depth = depth;
        this.ballCount = ballCount;
    }

    @Override
    public void W(int width) {
        this.width = width;
    }

    @Override
    public void L(int length) {
        this.length = length;
    }

    @Override
    public void D(int depth) {
        this.depth = depth;
    }

    public void displayBox() {
        System.out.println("\nBox Details");
        System.out.println("Width = " + width);
        System.out.println("Length = " + length);
        System.out.println("Depth = " + depth);
        System.out.println("Number of Balls = " + ballCount);
    }

    public static void main(String[] args) {

        Tray tray = new Tray(20, 30, 12);

        tray.displayTray();

        tray.W(25);
        tray.L(35);

        System.out.println("\nAfter Resizing Tray:");
        tray.displayTray();

        NewBox box = new NewBox(10, 15, 20, 50);

        box.displayBox();
```

```
        box.W(12);
        box.L(18);
        box.D(25);

        System.out.println("\nAfter Resizing Box:");
        box.displayBox();
    }
}
```

Output :

```
Tray Details
Width = 20
Length = 30
Number of Plates = 12

After Resizing Tray:

Tray Details
Width = 25
Length = 35
Number of Plates = 12

Box Details
Width = 10
Length = 15
Depth = 20
Number of Balls = 50

After Resizing Box:

Box Details
Width = 12
Length = 18
Depth = 25
Number of Balls = 50
```

Assignment 8:

- **Write the first 10 Fibonacci numbers to a text file "Fibo.txt".**
- **Read "Fibo.txt" and append next numbers (<250).**
- **Generate a binary file "Fibo.bin" from "Fibo.txt".**

Source Code :

```
import java.io.*;
import java.util.*;

public class FibonacciFile {

    public static void main(String[] args) {

        try {


            FileWriter fw = new FileWriter("Fibo.txt");

            int a = 0, b = 1;

            for (int i = 1; i <= 10; i++) {
                fw.write(a + " ");
                int c = a + b;
                a = b;
                b = c;
            }

            fw.close();

            System.out.println("First 10 Fibonacci numbers written to
Fibo.txt");


            Scanner sc = new Scanner(new FileReader("Fibo.txt"));

            ArrayList<Integer> fibo = new ArrayList<>();

            while (sc.hasNextInt()) {
                fibo.add(sc.nextInt());
            }

            sc.close();

            System.out.println("\nContents of Fibo.txt: "+fibo);

            FileWriter appendWriter = new FileWriter("Fibo.txt", true);

            int n1 = fibo.get(fibo.size() - 2);
            int n2 = fibo.get(fibo.size() - 1);

            int next = n1 + n2;
            while (next < 250) {
                appendWriter.write(next + " ");
```

```
                n1 = n2;
                n2 = next;
                next = n1 + n2;
            }

            appendWriter.close();

            System.out.println("\nNumbers appended successfully.");


            Scanner sc2 = new Scanner(new FileReader("Fibo.txt"));

            FileOutputStream fos = new FileOutputStream("Fibo.bin");

            while (sc2.hasNextInt()) {
                int num = sc2.nextInt();
                fos.write(num);
            }

            sc2.close();
            fos.close();

            System.out.println("Binary file Fibo.bin created.");
        }

        catch (IOException e) {
            System.out.println(e);
        }
    }
}
```

Output ;

```
First 10 Fibonacci numbers written to Fibo.txt

Contents of Fibo.txt: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

Numbers appended successfully.
Binary file Fibo.bin created.
```

Assignment 9:

- **Create a class Profile to store Name, Age, Average Marks. Allow creation of objects and store information for #n persons, with provision for storing content in "Profile.bin".**
- **Read "Profile.bin" and display the record for the highest scorer.**

Source Code :

```
import java.io.*;
import java.util.ArrayList;
import java.util.Random;

public class Profile implements Serializable {

    private String name;
    private int age;
    private float avgMarks;

    public Profile() {
        name = "";
        age = 0;
        avgMarks = 0;
    }

    public Profile(String name, int age, float avgMarks) {
        this.name = name;
        this.age = age;

        this.avgMarks = Math.round(avgMarks * 100) / 100.0f;
    }

    public float getAvgMarks() {
        return avgMarks;
    }

    @Override
    public String toString() {
        return String.format(
                "Name : %-20s Age : %2d  Average Marks : %.2f",
                name, age, avgMarks
        );
    }

    public static void main(String[] args) {

        String[] names = {
                "Ram", "Laxman", "Krishna",
                "Radha", "Sita", "Arjun",
                "Nakul", "Bhim", "Ajoy"
        };

        String[] titles = {
                "Sarkar", "Roy", "Chatterjee",
                "Ghosh", "Barman", "Mondal",
                "Banerjee"
```

```
        };

        ArrayList<Profile> profiles = new ArrayList<>();
        Random r = new Random();

        int n = r.nextInt(5, 15);

        System.out.println("Number of Profiles = " + n);

        for (int i = 0; i < n; i++) {

            String fullName = names[r.nextInt(names.length)]+ " "+
titles[r.nextInt(titles.length)];
            int age = r.nextInt(18, 26);
            float marks = r.nextFloat() * 100;

            profiles.add(new Profile(fullName, age, marks));
        }

        try {

            ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("Profile.bin"));

            for (Profile p : profiles) {
                oos.writeObject(p);
            }

            oos.close();

            System.out.println("\nProfiles successfully stored in Profile.bin");

            ObjectInputStream ois = new ObjectInputStream( new
FileInputStream("Profile.bin"));

            Profile highestScorer = null;

            System.out.println("\nAll Profiles");
            System.out.println("-----------------------------------------------
-----------");

            try {
                while (true) {
                    Profile p = (Profile) ois.readObject();
                    System.out.println(p);
                    if (highestScorer == null || p.getAvgMarks() >
highestScorer.getAvgMarks()) {
                        highestScorer = p;
                    }
                }

            } catch (EOFException e) {
```

```
            }

            ois.close();

            System.out.println("\nHighest Scorer");
            System.out.println("-----------------------------------------------
-----------");
            System.out.println(highestScorer);

        } catch (IOException | ClassNotFoundException e) {

            System.out.println("Error : " + e.getMessage());
        }
    }
}
```

Output :

```
Number of Profiles = 11

Profiles successfully stored in Profile.bin

All Profiles
------------------------------------------------------------
Name : Arjun Ghosh          Age : 19  Average Marks : 75.61
Name : Nakul Sarkar         Age : 23  Average Marks : 40.84
Name : Ajoy Barman          Age : 22  Average Marks : 96.92
Name : Ram Banerjee         Age : 22  Average Marks : 58.62
Name : Radha Ghosh          Age : 24  Average Marks : 50.01
Name : Ajoy Ghosh           Age : 18  Average Marks : 40.60
Name : Radha Barman         Age : 19  Average Marks : 56.75
Name : Krishna Banerjee     Age : 22  Average Marks : 12.26
Name : Radha Banerjee       Age : 25  Average Marks : 85.14
Name : Ajoy Mondal          Age : 22  Average Marks : 25.77
Name : Arjun Chatterjee     Age : 18  Average Marks : 91.54

Highest Scorer
------------------------------------------------------------
Name : Ajoy Barman          Age : 22  Average Marks : 96.92
```